Spring Boot Exam 3 – Marked Feedback

Nov 6, 2026 · Marked by @Anesu

Overview — 42 / 100 (does not meet the 60% pass mark)

Section	Marks available	Marks awarded
A — Setup, Validation, Error Handling	10	2
B — Entity Relationships	25	17
C — DTOs and JSON Responses	10	1
D — Relationship Endpoints	25	6
E — Custom Queries and Pagination	10	7
F — Evidence and Verification	10	0
G — Theory	10	9
Total	**100**	**42**

One root cause explains most of the lost marks: the controllers talk directly in JPA entities (Student, Department, Course, Enrollment) instead of the DTOs that were already written in `dto/`. That single gap cascades outward — `@Valid` can't run because the `@RequestBody` types don't carry the validation annotations, not-found/duplicate cases fall through to raw exceptions `GlobalExceptionHandler` never sees, and serializing an entity with a `@ManyToMany` on both sides risks the exact infinite-JSON failure the theory answers describe. Detailed, section-by-section marking follows below, and the same explanations are left as comments directly in the project's source files.

Section A — Setup, Validation and Error Handling: 2 / 10

Q1 — Prepare the project: 0 / 4

- `spring.datasource.url` points at `studenttt_relations_db` (typo'd), not the `student_relations_db` the spec names. (0/2)

- `docs/db/create_database.sql` does not exist, and `application.properties` commits a literal password (`notes`) instead of reading one from an environment variable. (0/2)

Q2 — Validation and consistent error responses: 2 / 6

- `GlobalExceptionHandler` is correctly built — the 400 field-error map, the 404 `{status,message}` body and the 409 `{status,message}` body all match the spec's shapes exactly. But `@Valid` is never added to a controller parameter anywhere in the project, so `MethodArgumentNotValidException` never fires — a blank name or `age: 200` is accepted as-is. (~1/3)
- `ResourceNotFoundException` and `DuplicateResourceException` are both defined and handled correctly, but neither is ever thrown — controllers use `new RuntimeException(...)` (which the handler doesn't catch, so it becomes an unhandled 500) or `ResponseEntity.notFound().build()` (404 with an empty body, not the documented JSON shape). (~1/3 across the two remaining marks)

See the comment left directly in `GlobalExceptionHandler.java` — the handler itself isn't the problem.

Section B — Entity Relationships: 17 / 25

Q3 — Department entity: 2 / 6 `id` and `name` (unique, not null) are correct. The `building` field (optional `String`) and the `students` `@OneToMany(mappedBy = "department")` field are both missing entirely. That second one matters beyond the 6 marks here: it's what `GET /api/departments/{id}/students` and the "still has students" check on delete would be built from (see Section D).

Q4 — Course entity: 4 / 6 `id`, `code` (unique), `title`, `credits`, and the inverse `students` `@ManyToMany(mappedBy = "courses")` are all correct. But two fields were added that the spec's relationship diagram doesn't include: `capacity` and a `@ManyToOne department`. The diagram is explicit — Department 1–* Student – Course, no Department–Course line — and making `department` NOT NULL means `POST /api/courses` now requires a `departmentId` the exam's own test script never sends (Section F, step 2 creates courses with just code/title/credits).

Q5 — Student entity: 11 / 13 This is the strongest part of the submission: the old `course` field is gone, `@ManyToOne(optional = false)` on `department_id` gives the required NOT NULL FK, and the `@ManyToMany` + `@JoinTable` builds `student_courses(student_id,`

`course_id)` with Student correctly as the owning side, using a `Set` as hinted. The 2 marks for confirming all four tables via `SHOW TABLES` weren't claimed — no evidence was submitted for this (see Section F).

Section C — DTOs and JSON Responses: 1 / 10

Q6 — Stop returning entities from controllers: 1 / 10 `StudentRequest`, `StudentResponse`, `CourseRequest`, `CourseResponse` and `DepartmentResponse` all exist in `dto/`, and `StudentResponse` even builds the exact nested shape the spec asks for (`department: {id, name}`, `courses: [...]`). But no controller uses any of them — `StudentController`, `DepartmentController` and `CourseController` all accept and return raw entities (or `Map<String, Object>`). Two consequences follow: the response JSON doesn't match the required shape, and there's a real risk of infinite recursion — `Student.courses` and `Course.students` are the two sides of the same `@ManyToMany`, so serializing a `Student` with any courses would serialize their `Department`, then `courses`, then each `Course`'s `students` set (which includes this student again), and so on. This is precisely the failure your own theory answer to Q2 describes — the DTOs that would prevent it are already written, just not wired into the controllers.

Credit: 1 mark for the DTO classes existing with mostly correct shapes and validation annotations.

Section D — Relationship Endpoints: 6 / 25

Endpoint	Marks	Awarded	Why
POST /api/departments	3	1	201 correct; no case-insensitive duplicate-name check → 500 instead of 409
GET /api/departments	1	1	Works
GET /api/departments/{id}/students	3	0	Not implemented
DELETE /api/departments/{id}	3	1	404/204 correct; no check for existing students → FK violation surfaces as 500, not 409

Endpoint	Marks	Awarded	Why
POST /api/courses	2	0	Requires non-spec departmentId + capacity; the spec's own example request (code/title/credits only) would be rejected; no duplicate-code check
GET /api/courses	1	1	Works
GET /api/courses/{id}/students	2	2	Implemented correctly (in EnrollmentController)
POST /api/students	3	0	Accepts a raw Student (needs a nested department object, not departmentId); returns 200 not 201; no 404/409 handling
POST /api/students/{id}/courses/{courseId}	3	0	Built instead as POST /api/enrollments with a different URL shape and a separate Enrollment entity disconnected from Student.courses; returns the raw enrollment, not student JSON
DELETE /api/students/{id}/courses/{courseId}	2	0	Built instead as DELETE /api/enrollments?studentId=&courseId=
GET /api/students/{id}/courses	2	0	Not implemented

Also missing: `PUT /api/students/{id}` doesn't exist at all — the spec says the Exam 2 GET/PUT/PATCH/DELETE endpoints must keep working, and only GET/PATCH/DELETE do.

Full explanations, with suggested fixes, are left as comments directly in `StudentController.java`, `DepartmentController.java`, `CourseController.java` and `EnrollmentController.java`.

Section E — Custom Queries and Pagination: 7 / 10

Derived query (lastName): 3 / 3 — `findByLastNameIgnoreCase` is correct and wired up to `GET /api/students/search?lastName=`.

JPQL @Query (courseCode): 2 / 4 — `StudentRepository.findByCourseCode` is a correct, hand-written JPQL join exactly matching the spec. It's just never called: `search()` in `StudentController` only reads a `lastName` parameter, so `?courseCode=DB201` isn't handled. The query itself needed no fixing, only a branch in the controller.

Pagination: 2 / 3 — `GET /api/students` correctly accepts a `Pageable` and returns `Page<Student>`; Spring Boot 3.x's built-in Jackson module serializes that to the `content` / `page.totalElements` shape the spec wants even without an explicit `new PagedModel<>(page)` wrap, so the shape mostly works. The mark lost here is the same DTO problem as Section C: the `content` array holds raw `Student` entities, not the required response shape, with the same recursion risk if any student has courses.

Section F — Evidence and Verification: 0 / 10

This section is marked on evidence only, and the spec says so explicitly: "Working code without the files below earns 0." `docs/exam3_evidence/` does not exist — no `requests.http`, no `SHOW CREATE TABLE` output, no `SELECT * FROM student_courses` snapshots, no screenshots for steps 5/11/12/13, and no `README.md` with setup/run instructions and an endpoint list. Even a fully correct implementation would score 0 here without these files, so this is worth prioritizing regardless of what else gets fixed.

Section G — Theory: 9 / 10

All five answers in `docs/theory_submission/exam_3_answers` are conceptually correct and clearly explained — owning side vs. `mappedBy`, why raw entities cause infinite JSON (and why DTOs fix it, which makes it worth noting the controllers don't actually apply that answer — see Section C), `LAZY` / `EAGER` and their defaults, why many-to-many needs a join table, and why 409 beats a raw 500. Full marks on content: 10/10.

One mark held back for the submission path: the spec asks for the file at `docs/theory_submission/exam3_answers.md` (with the `.md` extension). The actual file is `docs/theory_submission/exam_3_answers` — close, but an automated or scripted check looking for the exact filename wouldn't find it.

What to fix first

In priority order, by marks recoverable per unit of effort:

1. **Wire the existing DTOs into the controllers.** `StudentRequest` / `StudentResponse` / `CourseRequest` / `CourseResponse` / `DepartmentResponse` already exist and are mostly correct — swapping `@RequestBody Student` for `@RequestBody @Valid StudentRequest` (and returning `new StudentResponse(...)`) fixes validation (Section A), the response shape and recursion risk (Section C), and several Section D line items, all at once.
2. **Throw the exceptions `GlobalExceptionHandler` already handles correctly.** Replace `new RuntimeException(...)` and `ResponseEntity.notFound().build()` with `throw new ResourceNotFoundException(...)` / `throw new DuplicateResourceException(...)` wherever a lookup can fail or a unique constraint could be hit. The handler itself needs no changes.
3. **Add the two missing fields to `Department`** (`building`, and the `@OneToMany students` list) — this unblocks `GET /api/departments/{id}/students` and the 409-on-delete-with-students check in one move.
4. **Move enrol/unenrol/list-courses onto `StudentController` under the exact URLs the spec gives** (`/api/students/{id}/courses/{courseId}`, `/api/students/{id}/courses`), built on `Student.courses` directly rather than the separate `Enrollment` / `enrollments` table, so it agrees with the rest of the app.
5. **Remove (or make optional) the non-spec `department` / `capacity` fields on `Course`** so `POST /api/courses` accepts the plain `{code, title, credits}` body the spec's test script sends.
6. **Fix the database name and stop committing the real password**, add `docs/db/create_database.sql`, and start collecting the Section F evidence (`requests.http`, `SHOW CREATE TABLE` output, the three `student_courses` snapshots, screenshots, README) — this section is worth 10 marks and currently scores 0 regardless of code quality.
7. Rename `docs/theory_submission/exam_3_answers` to `exam3_answers.md`.

What was done well

Worth saying plainly: the hardest, most error-prone part of this exam — the JPA relationship mappings on `Student` (owning side, NOT NULL FK, correct join table, `Set` not `List`) — is essentially correct, which is genuinely the part beginners get wrong most

often. The `GlobalExceptionHandler` is well-built and matches the spec's JSON shapes exactly. The theory answers show real understanding, including correctly diagnosing the infinite-JSON problem that the controllers themselves still have. The gap between this submission and a passing one isn't a lack of understanding — it's that the DTOs, exceptions and entity fields that were already written correctly in isolation were never connected to the controllers that needed them. That's a wiring problem, not a comprehension problem, and it's the fastest kind of gap to close.